CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL – UDF

COORDENAÇÃO DO CURSO DE SISTEMAS DE INFORMAÇÃO

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

BRASÍLIA

2026

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

Trabalho de conclusão de curso apresentado à Coordenação do Curso de Sistemas de Informação, do Centro Universitário do Distrito Federal - UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em Sistemas de Informação.

Orientador: Prof. Dr. Xxxx

BRASÍLIA

?Sobrenome?, ?Nome?. Título : ?author?. – Brasília, 2026. 28 f.

Orientador: ?orientador?. Trabalho de Conclusão de Curso (Graduação – Sistemas de Informação) – Centro Universitário do Distrito Federal – UDF. Coordenação do Curso de Sistemas de Informação, Brasília, DF, 2026.

1. ?Assunto?. 2. ?Assunto?. 3. ?Assunto?. I. ?Título?.

CDU: ?consultar na biblioteca?

?Titulo?

Trabalho de conclusão de curso apresentado à ?Coordenação?, do Centro Universitário do Distrito Federal – UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em ?Curso?.

Orientador: ?orientador?.

Brasília, _____ de _____ de 2026.

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

Nota: _____

Dedico este trabalho à minha família, pelo apoio, incentivo e compreensão durante toda a minha trajetória acadêmica.

Agradecimentos

Agradeço, primeiramente, a Deus, por mais uma conquista; ao meu orientador, pela dedicação e correções; e aos bibliotecários pelo suporte em todas as pesquisas.

"A educação é a arma mais poderosa que você pode usar
para mudar o mundo."

Nelson Mandela

Resumo

Este trabalho trata da atualização de sistemas legados para novas tecnologias com o uso de agentes de IA para desenvolvimento. O objetivo é identificar os desafios desse uso, do ponto de vista do desenvolvimento. Para isso, foi feita uma revisão de estudos recentes sobre o tema, e os desafios encontrados foram organizados em categorias. Também é proposto um estudo de caso em um sistema real, para observar esses desafios na prática, com avaliação por testes.

Palavras-chave: *Sistemas legados. Modernização de software. Agentes de IA. Desenvolvimento de software.*

Abstract

This work addresses the update of legacy systems to new technologies using AI development agents. The objective is to identify the challenges of this use from a development perspective. To do so, a review of recent studies on the subject was carried out, and the challenges found were organized into categories. A case study on a real system is also proposed, to observe these challenges in practice, with evaluation through tests.

Keywords: *Legacy systems. Software modernization. AI agents. Software development.*

Lista de ilustrações

Sumário

1 INTRODUÇÃO 12

2 OBJETIVOS 14
2.1 Objetivo geral 14
2.2 Objetivos específicos 14
2.3 Delimitação do trabalho 15
2.4 Contribuição esperada 15

3 FUNDAMENTAÇÃO TEÓRICA 16
3.1 Sistemas legados e modernização 16
3.2 Agentes de IA no desenvolvimento 17
3.3 Conversão de código e verificação 17
3.4 Desafios encontrados na literatura 18

4 METODOLOGIA 20
4.1 Tipo de pesquisa 20
4.2 O estudo de caso 20
4.3 Descrição do sistema atual 20
4.3.1 Finalidade e usuários 20
4.3.2 Tecnologia e estrutura 21
4.3.3 Fluxo de um pedido 21
4.3.4 Regras do sistema 21
4.3.4.1 Registros excluídos 21
4.3.4.2 Diárias nacionais por cargo 22
4.3.4.3 Valor de cada linha de custo 22
4.3.4.4 Total previsto de custos 22
4.3.5 Como as regras aparecem no código 23
4.4 Dados sintéticos 23
4.5 Processamento dos dados 23
4.6 Uso do agente de IA 24
4.7 Como será a avaliação 24

5 RESULTADOS E CONSIDERAÇÕES FINAIS 26

6 CONCLUSÃO 27

REFERÊNCIAS 28

1 Introdução

Sistemas legados são sistemas que continuam em uso nas organizações, mesmo quando suas tecnologias ou ferramentas já não acompanham as práticas atuais. Isso não depende só da idade do sistema. Depende também da importância dele para os processos da organização e da dificuldade de mantê-lo, evoluí-lo e integrá-lo a novas soluções.

Muitas vezes, esses sistemas guardam regras de negócio criadas e alteradas ao longo de anos. Parte desse conhecimento pode estar só no código, principalmente quando a documentação é incompleta ou não acompanha as mudanças. Atualizar um sistema legado para tecnologias mais novas é o que se chama de modernização. Ela não é só reproduzir as funcionalidades em uma tecnologia recente: também é preciso entender e preservar as regras importantes para a organização. O sistema novo precisa continuar fazendo o que o antigo fazia. Por isso, esse trabalho exige análise e validação cuidadosas.

Estudos recentes trazem exemplos desse cenário. Alves, Politowski e Montandon (2026) trabalharam com um sistema de gestão de shopping centers escrito em Visual Basic 6, uma linguagem antiga. O sistema estava em produção havia mais de vinte anos. Ele era organizado em um núcleo e em módulos secundários, acrescentados conforme o pedido de cada cliente da empresa dona do sistema. Os autores escolheram dois módulos de naturezas diferentes. O de Medições é voltado a cálculos numéricos, como consumo e tarifas. O de Compras depende de transações e de fluxos de aprovação. Os dois têm cerca de 299 mil e 224 mil linhas de código e são usados continuamente pelos clientes da empresa. Eles foram migrados para C# (.NET 10), mantendo o mesmo banco de dados. A migração foi feita por um agente de IA, em uma única geração e sem feedback humano entre as tentativas. Ela envolveu 12 funcionalidades, seis de cada módulo: 6 simples, 4 médias e 2 complexas. O objetivo era que o código novo se comportasse como o original.

O segundo exemplo vem da IBM. De acordo com Chakaravarthy et al. (2026), sistemas empresariais modernos muitas vezes dependem de aplicações antigas em COBOL, uma linguagem criada há várias décadas. A estrutura rígida e monolítica e as variáveis globais dessas aplicações dificultam a manutenção e a adoção da nuvem. Para lidar com isso, os autores propõem traduzir o COBOL para Java combinando a análise do código com um modelo de linguagem. A análise infere as classes e as assinaturas dos métodos em Java a partir dos dados e do fluxo de controle do programa em COBOL. Essas informações guiam o modelo na tradução da lógica dos programas. A solução faz parte de um produto da IBM, o watsonx Code Assistant for Z. Os autores a avaliaram em 20 programas de um conjunto chamado GenAPP e em 10 aplicações de clientes. Segundo eles, ela já é usada em vários ambientes de clientes da IBM.

Esses estudos também relatam dificuldades para o desenvolvedor. No estudo de Alves, Politowski e Montandon (2026), as regras espalhadas em vários arquivos e as regras escondidas em parâmetros opcionais e em configurações globais foram as mais difíceis de migrar. Outro estudo avaliou o uso de modelos de linguagem para documentar código em linguagens antigas, como MUMPS e assembly (DIGGS et al., 2024), para ajudar a entender esse tipo de código.

Uma forma de apoiar esse trabalho é usar agentes de IA para desenvolvimento. Eles são ferramentas que leem o código de um projeto, editam arquivos e rodam comandos e testes. Estudos recentes avaliaram esses agentes em tarefas de modernização (LI et al., 2026; LIU et al., 2026; MAY et al., 2025).

Os resultados desses estudos relatam limites. No estudo de Li et al. (2026), os agentes passaram em 91,3% dos testes em projetos pequenos, mas em apenas 15,3% nos projetos grandes. No estudo de Alves, Politowski e Montandon (2026), o agente deixou de migrar parte das regras do sistema, e as funcionalidades mais complexas foram as mais afetadas. Por isso, o desenvolvedor precisa saber quais desafios podem aparecer quando um agente de IA atualiza um sistema legado e como lidar com eles.

Este tema é relevante porque sistemas antigos precisam ser atualizados, mas esse trabalho é lento e arriscado. Um agente de IA pode acelerá-lo, mas a literatura relata desafios, como regras que o agente não percebe, atalhos que ele toma e resultados difíceis de verificar. Nos estudos que lemos, a avaliação usa conjuntos de testes públicos ou sistemas de empresas, e encontramos pouco sobre o uso do agente no dia a dia do desenvolvedor, em um sistema real e menor. Este trabalho busca ajudar o desenvolvedor a saber o que esperar e como se proteger. Para isso, faz uma revisão da literatura e propõe um estudo de caso em um sistema real, com avaliação por testes.

Este trabalho busca responder à seguinte pergunta:

> Quais são os desafios para converter e atualizar sistemas legados para novas tecnologias por meio de agentes de IA, do ponto de vista do desenvolvimento?

Este trabalho está organizado em seis capítulos, contando esta introdução. O Capítulo 2 apresenta os objetivos. O Capítulo 3 reúne o que a literatura relata sobre sistemas legados, agentes de IA, verificação do código e os desafios desse uso. Com base nisso, o Capítulo 4 descreve como o estudo de caso será feito, e o Capítulo 5 apresenta os resultados e as considerações finais. O Capítulo 6 traz a conclusão.

2 Objetivos

Este capítulo apresenta o que o trabalho pretende alcançar e o que fica fora dele. Como visto na introdução, os estudos sobre agentes de IA na modernização de sistemas legados relatam resultados que variam bastante, e há pouco registro do uso desses agentes no dia a dia do desenvolvedor. Os objetivos a seguir mostram como este trabalho busca responder à pergunta de pesquisa.

2.1 OBJETIVO GERAL

O objetivo geral deste trabalho é identificar e analisar os desafios de atualizar sistemas legados com a ajuda de agentes de IA, do ponto de vista do desenvolvimento. Ou seja, o foco está no trabalho de quem desenvolve: o que exige cuidado, o que pode dar errado e como conferir o resultado do agente.

Para isso, o trabalho parte de duas perguntas menores. A primeira é o que a literatura já relata sobre os desafios desse uso. A segunda é o que aparece quando esse uso é observado em um sistema real e pequeno.

2.2 OBJETIVOS ESPECÍFICOS

O primeiro objetivo específico é levantar na literatura os desafios já relatados. Para isso, o trabalho reúne estudos recentes que usaram agentes de IA ou modelos de linguagem em tarefas de modernização, como a conversão de linguagem e a atualização de versão. De cada estudo, são anotados o que foi testado, os resultados relatados e as dificuldades que apareceram. Esse levantamento está no Capítulo 3.

O segundo é organizar esses desafios em categorias. Os estudos usam sistemas, linguagens e agentes diferentes, e por isso as dificuldades aparecem de formas diferentes. Agrupar as parecidas, como tamanho do sistema, regras escondidas e atalhos do agente, facilita a comparação e serve de base para o estudo de caso. O resultado é o quadro de desafios, também no Capítulo 3.

O terceiro é observar esses desafios em um estudo de caso. O trabalho propõe atualizar partes de um sistema real com a ajuda de um agente de IA, usando dados inventados. O resultado de cada parte é avaliado por testes que comparam o sistema novo com o atual, e cada problema

encontrado é registrado e classificado nas categorias do objetivo anterior. O método está no Capítulo 4, e os resultados, no Capítulo 5.

O quarto é discutir o que o desenvolvedor pode fazer para lidar com cada desafio. Com base na literatura e no estudo de caso, o trabalho aponta cuidados práticos, como atualizar em partes pequenas, proteger os testes e conferir se o agente tomou atalhos. Essa discussão aparece nos Capítulos 5 e 6.

2.3 DELIMITAÇÃO DO TRABALHO

O trabalho tem limites que ajudam a entender o que ele oferece. O estudo de caso observa um sistema, um agente e uma troca de tecnologia. Por isso, os resultados mostrarão o que aconteceu naquele caso e não valerão para todos os sistemas. O trabalho também não compara agentes entre si, não mede o custo financeiro em detalhe e não avalia a qualidade do código além dos testes e da revisão do desenvolvedor.

2.4 CONTRIBUIÇÃO ESPERADA

Espera-se que o trabalho ajude o desenvolvedor a decidir quando usar um agente de IA na modernização de um sistema legado e quais cuidados tomar. Espera-se também deixar um método simples, que possa ser repetido em outros sistemas, como indicado nos trabalhos futuros.

3 Fundamentação Teórica

Este capítulo apresenta a base teórica do trabalho: o que são sistemas legados e modernização, como agentes de IA são usados nessa tarefa, como o resultado é verificado e quais desafios a literatura relata.

Esta revisão reúne estudos publicados entre 2024 e 2026 sobre modernização de sistemas legados com agentes de IA. A busca foi feita no Google e no Google Acadêmico, e teve o apoio de uma ferramenta de IA. Foram escolhidos os estudos que tratam de migração, atualização ou documentação de código legado com o uso de agentes ou modelos de linguagem e que relatam resultados ou dificuldades. Esta não é uma revisão sistemática, por isso pode não incluir todos os estudos da área.

3.1 SISTEMAS LEGADOS E MODERNIZAÇÃO

Um sistema legado é um sistema antigo que ainda está em uso. Ele pode estar escrito em uma linguagem pouco usada hoje, ter uma estrutura difícil de mudar ou depender de bibliotecas sem suporte. Mesmo assim, ele pode ser importante, porque muitas vezes guarda regras de negócio que a organização usa todos os dias.

Um exemplo é o sistema estudado por Alves, Politowski e Montandon (2026). Era um sistema de gestão em Visual Basic 6, em produção havia mais de vinte anos. Outro exemplo são os programas em COBOL, que a IBM migra para Java (CHAKARAVARTHY et al., 2026).

A modernização é o trabalho de levar esse sistema para uma tecnologia mais nova, sem perder o que ele já faz. Ela pode ser de dois tipos:

- **conversão de linguagem**, por exemplo de Visual Basic 6 para C#;
- **atualização de versão**, por exemplo de Java 8 para Java 17 (LIU et al., 2026; MAY et al., 2025).

Uma dificuldade relatada é a falta de documentação. Em alguns casos, o código é a única fonte do que o sistema faz. Diggs et al. (2024) testaram modelos de linguagem para documentar código legado em MUMPS e em assembly. Entender o código antigo é uma etapa importante da modernização.

3.2 AGENTES DE IA NO DESENVOLVIMENTO

Entender e atualizar um sistema legado dá trabalho. Agentes de IA são uma forma de apoiar esse trabalho. Um agente de IA para desenvolvimento é uma ferramenta baseada em um modelo de linguagem que faz mais do que responder perguntas. Ele lê os arquivos do projeto, edita o código, roda comandos e testes, e repete esses passos até chegar a um resultado.

Na modernização, o agente pode receber o código antigo e a tecnologia nova como alvo, e propor o código novo. O desenvolvedor trabalha junto: define a tarefa, revisa o resultado e decide o que aceitar.

Os agentes já foram testados nesse tipo de tarefa. Alves, Politowski e Montandon (2026) usaram o Claude Code para migrar módulos de um sistema em Visual Basic 6 para C#. May et al. (2025) e Liu et al. (2026) avaliaram agentes na atualização de projetos Java. Li et al. (2026) avaliaram agentes na modernização de repositórios em várias linguagens.

Outra linha de trabalho junta a IA com análise do código. A IBM combina análise estática do código COBOL com um modelo de linguagem para gerar código Java (CHAKARAVARTHY et al., 2026). A análise dá ao modelo informações sobre a estrutura do programa, e, segundo os autores, o resultado ficou melhor do que o de modelos usados sozinhos.

3.3 CONVERSÃO DE CÓDIGO E VERIFICAÇÃO

Quando um agente ou um modelo de linguagem gera código novo, é preciso conferir o resultado. Converter código não é só trocar a sintaxe. O código novo precisa fazer a mesma coisa que o antigo. Isso é chamado de equivalência funcional.

Por isso, Pan et al. (2024) consideram uma tradução correta só quando o código compila, roda e passa nos testes. Compilar não basta, porque um código pode compilar e mesmo assim se comportar de forma diferente.

Os testes também têm limites. Se o sistema tem poucos testes, erros podem passar sem ser vistos. No estudo de May et al. (2025), os projetos escolhidos têm alta cobertura de testes, e a cobertura não pode cair mais de 5 pontos percentuais depois da migração. No estudo de Li et al. (2026), os testes ficam escondidos do agente, para que ele não os use como atalho.

Outra forma de avaliar é pedir para outro modelo de linguagem julgar o resultado. A IBM usou esse método para medir a qualidade do código Java gerado (CHAKARAVARTHY et al., 2026). Mas, no estudo de Diggs et al. (2024), as métricas automáticas tiveram pouca relação com a nota dada por pessoas, e por isso os autores indicam a avaliação humana. Neste trabalho, a

verificação principal será feita com testes.

3.4 DESAFIOS ENCONTRADOS NA LITERATURA

Além da verificação, há outros pontos a observar. A literatura relata resultados bons no uso de agentes de IA na modernização, mas também desafios. Eles estão resumidos no Quadro 1 e explicados a seguir.

Quadro 1 – Desafios da modernização com agentes de IA

Desafio	**O que os estudos relatam**	**Fonte**
Tamanho do sistema	Passou em 91,3% dos testes com menos de 10 mil linhas e em 15,3% com mais de 50 mil	Li et al. (2026)
Complexidade das funcionalidades	Equivalência de 92% (baixa), 81% (média) e 47% (alta)	Alves, Politowski e Montandon (2026)
Regras escondidas	O agente perdeu regras implícitas e regras espalhadas em vários arquivos	Alves, Politowski e Montandon (2026)
Atalhos do agente	Excluiu testes que falhavam e executou o programa original em vez de traduzi-lo	May et al. (2025); Li et al. (2026)
Linguagens antigas	Bons resultados em MUMPS, notas ruins em assembly, e alucinações em ambos	Diggs et al. (2024)
Custo	Em média, 3,81 milhões de tokens e US$ 4,19 por migração	Alves, Politowski e Montandon (2026)

Fonte: elaborado pela autora, com base nos trabalhos citados.

Tamanho e complexidade

Nos estudos lidos, os resultados foram piores em sistemas maiores e mais complexos. Além dos números do Quadro 1, Liu et al. (2026) relataram 71,67% de sucesso na migração mínima de Java 8 para Java 17 e 53,33% na migração máxima. No estudo de May et al. (2025), o melhor modelo migrou 52,3% dos projetos.

Regras que o agente não vê

No estudo de Alves, Politowski e Montandon (2026), o agente deixou de migrar 100 de 331 itens analisados. A maior causa, com 42 casos, foram módulos vizinhos que ele não detectou. As regras escritas de forma explícita foram migradas melhor do que as implícitas, como as que dependem de parâmetros opcionais ou de configurações globais.

Atalhos e falsa impressão de sucesso

O agente pode parecer ter acertado sem ter acertado. No estudo de May et al. (2025), um agente excluiu testes que falhavam do processo de construção do projeto. No estudo de Li et al. (2026), em uma execução sem regras contra atalhos, um agente chegou a 99,5% de testes aprovados, mas em cinco projetos ele só chamava o programa original. Por isso, o desenvolvedor deve proteger os testes e conferir o que o agente de fato fez.

Linguagens antigas e pouca documentação

Diggs et al. (2024) observaram que os modelos tendem a inventar informações em código legado. Os comentários gerados para MUMPS foram bem avaliados, mas os de assembly tiveram notas baixas.

Custo

O custo depende do tamanho da tarefa. No estudo de Alves, Politowski e Montandon (2026), as funcionalidades de alta complexidade usaram cerca de 9 milhões de tokens, em média, e a maior parte do gasto vem do que o agente lê, e não do que escreve.

Supervisão do desenvolvedor

Nos estudos lidos, o resultado do agente foi verificado de alguma forma. Alves, Politowski e Montandon (2026) usaram um segundo revisor para conferir as regras migradas. Diggs et al. (2024) indicam avaliação humana, porque as métricas automáticas não bastaram. E Chakaravarthy et al. (2026) relatam que juntar o modelo com análise do código melhorou os resultados e reduziu a variação entre eles.

O que ainda falta

Nos estudos que lemos, a avaliação usa conjuntos de testes públicos ou sistemas de empresas, e encontramos pouco sobre o uso do agente no dia a dia do desenvolvedor, em um sistema real e menor. É isso que o estudo de caso deste trabalho propõe.

4 Metodologia

Este capítulo descreve como o estudo de caso será feito.

4.1 TIPO DE PESQUISA

Este trabalho tem abordagem qualitativa e caráter exploratório. Ele tem duas partes: uma revisão da literatura, apresentada no Capítulo 3, e um estudo de caso, descrito neste capítulo. A pesquisa é qualitativa porque descreve e analisa os desafios encontrados, sem medir com estatística. É exploratória porque busca conhecer melhor um tema ainda pouco observado na prática do desenvolvedor.

4.2 O ESTUDO DE CASO

Para observar na prática os desafios relatados na literatura, este trabalho propõe um estudo de caso com um sistema real. O sistema escolhido é o SIS Eventos, usado para registrar e aprovar a participação de pessoas em eventos. Ele foi feito no Microsoft Power Apps, com dados em listas do SharePoint, e será atualizado para Python, com testes automáticos.

Serão atualizadas três regras do sistema, com graus de complexidade diferentes: o tratamento dos registros excluídos (simples), o valor de cada linha de custo (média) e o total previsto de custos (complexa). Essas regras estão descritas na seção seguinte.

O responsável pelo sistema é o *stakeholder* do trabalho. Ele autoriza o uso do sistema, explica as regras importantes e ajuda a conferir se o resultado está correto.

4.3 DESCRIÇÃO DO SISTEMA ATUAL

4.3.1 Finalidade e usuários

O SIS Eventos é usado em uma instituição pública para registrar e aprovar a participação de pessoas em eventos. Quem tem interesse em ir a um evento pela instituição cadastra um pedido no sistema, com dados como o evento, a data, a passagem e os valores. O gabinete usa o sistema

para registrar e acompanhar esses eventos, e a aprovação é feita pelo presidente. O sistema também tem cadastros de áreas requisitantes, de pontos focais por área e de permissões dos usuários.

4.3.2 Tecnologia e estrutura

O sistema foi feito no Microsoft Power Apps e guarda os dados em listas do SharePoint. Ele tem nove telas: um painel de eventos por ano e por status, a lista de solicitações, o formulário para criar uma solicitação, as pendências, a prestação de contas, a configuração e os três cadastros. Os dados ficam em dez listas, como eventos, países, cargos, diárias nacionais, diárias internacionais e previsão de custos dos eventos.

4.3.3 Fluxo de um pedido

O pedido começa com o cadastro do evento. A pessoa informa dados como identificação, organizador, datas, tipo de evento, âmbito (nacional ou internacional), local, área requisitante, número do processo e justificativa. Junto com o pedido, ela registra os custos que espera ter, como passagem, diárias, seguro viagem, inscrição e custos complementares. O sistema guarda quem criou o pedido e quem fez a última alteração, com data e hora.

Cada pedido tem um status: Em Aberto, Em Análise, Aprovado, Não Aprovado, Cancelado ou Aprovado para ajustes. Neste último, o pedido volta para a pessoa corrigir. Depois da aprovação, os valores são pagos. Quando o evento termina, a prestação de contas compara o que foi previsto com o que foi realmente gasto.

4.3.4 Regras do sistema

Esta seção descreve as regras que foram lidas no código do sistema, no editor do Power Apps. Elas são a base para os testes do estudo de caso.

4.3.4.1 Registros excluídos

As listas têm um campo chamado Excluido. Os eventos, os custos e as diárias marcados com um asterisco (*) nesse campo são ignorados nas listas de escolha e nas contas.

4.3.4.2 Diárias nacionais por cargo

Em um evento nacional, a pessoa escolhe o cargo ou função. O sistema então mostra as diárias da lista de diárias nacionais que têm a mesma classificação desse cargo. Cada opção aparece com o nome da cidade ou capital e o valor da diária. A classe do cargo é preenchida automaticamente a partir da lista de cargos. Se o âmbito do evento não for nacional, essa lista não mostra opções.

4.3.4.3 Valor de cada linha de custo

O valor de cada linha de custo depende do tipo de custo, como mostra o Quadro 2. O sistema testa os casos em ordem e usa o primeiro que se aplica. O resultado aparece em reais.

Quadro 2 – Cálculo do valor de cada linha de custo

Tipo de custo	**Como o valor é calculado**
Seguro viagem	valor do seguro × quantidade de pessoas
Passagem	quantidade de pessoas × passagem individual
Diária internacional	valor já convertido para reais, guardado na linha
Diária nacional (da lista)	quantidade de diárias × valor da diária
Diária de outro cargo (valor digitado)	quantidade de diárias × valor digitado × quantidade de pessoas
Inscrição	valor já convertido para reais, guardado na linha
Custo complementar	o próprio valor

Fonte: elaborado pela autora, com base no código do sistema.

4.3.4.4 Total previsto de custos

O sistema calcula o total previsto de custos dos eventos aprovados de um ano. Para cada evento aprovado, ele soma as linhas de custo previstas que não foram excluídas. Cada linha soma sete partes:

- diária de outro cargo: quantidade de diárias × valor digitado × quantidade de pessoas;
- diária nacional: quantidade de diárias × valor da lista × quantidade de pessoas;
- diária internacional: cotação do dólar × valor em dólar × quantidade de diárias × quantidade de pessoas;

- inscrição: o valor, multiplicado pela cotação do dólar quando a moeda é o dólar;
- custo complementar: o próprio valor;
- seguro viagem: valor × quantidade de pessoas;
- passagem: passagem individual × quantidade de pessoas.

Cada parte é arredondada para duas casas decimais, e o total aparece em reais. Essa conta soma todos os campos da linha, sem olhar o tipo de custo.

4.3.5 Como as regras aparecem no código

As contas ficam dentro de controles das telas, como galerias, botões e campos de texto, e não em um lugar só. A busca no editor encontrou 39 ocorrências da lista de diárias nacionais e 88 da quantidade de pessoas, em várias telas. A fórmula que monta a lista de diárias nacionais tem três casos com as mesmas contas e condições diferentes. O editor também avisa que há controles com versões mais recentes disponíveis, como 20 controles na tela de exercícios. Esses pontos serão observados no estudo de caso.

4.4 DADOS SINTÉTICOS

O estudo de caso usará dados sintéticos, ou seja, dados inventados para o estudo. Nenhum dado real será usado. Dados sensíveis e senhas não fazem parte do estudo. Os dados inventados terão os mesmos campos das listas do sistema atual, como eventos, cargos, diárias e custos. Os casos de teste serão escolhidos para passar por cada regra da descrição do sistema atual, como os registros excluídos, cada tipo de custo e o total previsto.

4.5 PROCESSAMENTO DOS DADOS

No sistema atual, os dados ficam em listas do SharePoint, e as contas ficam dentro de controles das telas do Power Apps, e não em um lugar só. Cada caso de teste será executado no sistema atual, e o resultado será anotado, como o valor de cada linha de custo e o total previsto.

No sistema novo, em Python, os mesmos dados serão processados pelo código feito com a ajuda do agente. O resultado de cada caso será comparado com o do sistema atual. Se os resultados forem iguais, a parte atualizada é considerada equivalente nesse caso.

4.6 USO DO AGENTE DE IA

A atualização será feita com um agente de IA, que trabalha junto com o desenvolvedor, dentro do ambiente de desenvolvimento. O processo tem cinco passos:

1. entender o sistema: o agente lê o código e explica o que cada parte faz;
2. criar os testes antes de mudar o código, com base nos resultados do sistema atual;
3. pedir a atualização em partes pequenas, e não o sistema inteiro de uma vez;
4. rodar os testes depois de cada parte;
5. revisar o resultado e decidir o que aceitar.

O código só será usado com o agente se o responsável pelo sistema autorizar por escrito. Se o código não puder sair do ambiente da instituição, o estudo será feito apenas com a parte autorizada.

4.7 COMO SERÁ A AVALIAÇÃO

Cada parte atualizada só é considerada correta se o código compilar, rodar e passar nos testes (PAN et al., 2024), ou seja, se der os mesmos resultados do sistema atual nos casos de teste. Além disso:

- os testes ficam sob controle do desenvolvedor, e o agente não pode apagá-los ou mudá-los;
- os testes devem cobrir cada regra da descrição do sistema atual, e esse conjunto não pode diminuir depois da atualização;
- o desenvolvedor confere se o agente não tomou atalhos, como chamar o código antigo em vez de reescrevê-lo.

Durante o trabalho, cada problema encontrado é registrado e classificado de acordo com os desafios do Quadro 1. Também são anotados o tempo gasto e o uso do agente, para ter uma ideia do custo.

Este estudo tem limites. Ele observa um sistema, um agente e uma troca de tecnologia, então os resultados mostrarão o que aconteceu nesse caso e não valerão para todos os sistemas. Estudos parecidos, como o de Alves, Politowski e Montandon (2026), também tratam seus resultados

dessa forma. O agente pode dar resultados diferentes a cada execução, e a mesma pessoa conduz e avalia o estudo. Além disso, o sistema atual foi feito em Power Apps, e não em código escrito em arquivos, como os sistemas dos estudos citados.

5 Resultados e Considerações Finais

6 Conclusão

Este trabalho teve como objetivo entender os desafios de atualizar sistemas legados com agentes de IA, do ponto de vista do desenvolvimento.

Na literatura, encontramos desafios em seis pontos: tamanho do sistema, complexidade das funcionalidades, regras escondidas, atalhos do agente, linguagens antigas e custo. Nos estudos lidos, o resultado do agente foi verificado de alguma forma, e os testes foram uma das formas usadas para isso.

O estudo de caso foi proposto para observar esses desafios em um sistema real, o SIS Eventos. A proposta é usar o agente em partes pequenas, proteger os testes, comparar os resultados com os do sistema atual e registrar cada problema encontrado.

Como trabalhos futuros, o mesmo método pode ser aplicado em outros sistemas, com outros agentes e outras trocas de tecnologia, para comparar os resultados.

Referências

ALVES, I. d. S. R.; POLITOWSKI, C.; MONTANDON, J. E. Legacy System Modernization with Coding Agents: A Case Study. 2026. ArXiv:2608.28972. Disponível em: <https://arxiv.org/abs/2608.28972>.

CHAKARAVARTHY, V. et al. Enterprise-scale COBOL-to-Java translation: LLMs augmented with program analysis. In: Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26). Rio de Janeiro, Brasil: [s.n.], 2026.

DIGGS, C. et al. Leveraging LLMs for Legacy Code Modernization: Challenges and Opportunities for LLM-Generated Documentation. 2024. ArXiv:2411.14971. Disponível em: <https://arxiv.org/abs/2411.14971>.

LI, X. et al. RepoMod-Bench: A Benchmark for Code Repository Modernization via Implementation-Agnostic Testing. 2026. ArXiv:2602.22518. Disponível em: <https://arxiv.org/abs/2602.22518>.

LIU, L. et al. MigrationBench: Repository-level code migration benchmark from java 8. In: Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '26). Jeju, Coreia do Sul: [s.n.], 2026. ArXiv:2505.09569.

MAY, V. et al. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. 2025. ArXiv:2510.04852. Disponível em: <https://arxiv.org/abs/2510.04852>.

PAN, R. et al. Lost in translation: A study of bugs introduced by large language models while translating code. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24). Lisboa, Portugal: [s.n.], 2024.